

Theory

Semester II

60 contact hours

4 credits

CIE 40 / ESE 60

2132

Programming in C

Structured programming in C: language basics, control flow, arrays, strings, functions, structures, pointers and file handling.

Course code	2132
Scheme	4:0:0:0
Credits	4
Contact hours	60
Self-learning	6 / week; 90 h listed
Type	Theory

OFFICIAL SOURCE DECLARATION

How this lesson was prepared

The supplied official syllabus PDF was used as the authoritative source for course information, objectives, course outcomes, CO-PO mapping, modules, topics, instructional hours, self-learning activities, assessment structure and listed learning resources. Educational explanations, examples and diagrams have been added to make the official topics learnable; they are not presented as additional official syllabus requirements.

Source file: 2132.pdf

Curriculum: Diploma Curriculum Revision 2026.

The CO mentions dynamic memory allocation, while the detailed syllabus table explicitly lists pointers and file operations without a separate dynamic-memory subtopic. Pointer/file explanations are included; dynamic allocation is noted as an outcome-related extension, not falsely represented as a separate official major topic.

AT A GLANCE

Course dashboard

Course dashboard	
Field	Official information
Programme	AI; AI & ML; Cyber Forensics; CSE; Computer Engineering; IT; RPA and listed related programmes
Course code / title	2132 — Programming in C
Semester / type	II / Theory
Teaching scheme	4:0:0:0 (L:T:P:R)
Contact hours / credits	60 / 4
Self-learning	6 per week; total 90 hours printed
Assessment	CIE 40; ESE 60
Prerequisite	Course 1131 Problem solving and Python programming, Semester I

STUDY METHOD

How to use this lesson

Before class

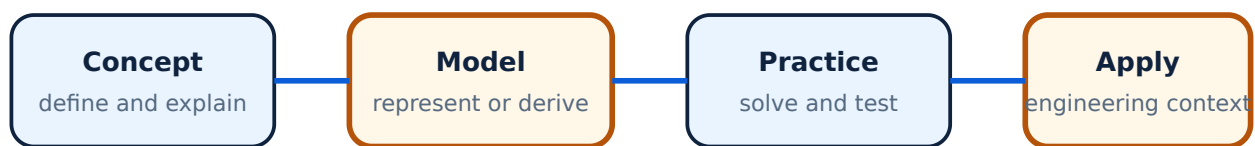
Read the topic definition and inspect the diagram. Mark unfamiliar technical words in the glossary.

During practice

Reproduce the worked example or procedure without looking at the answer. For theory topics, redraw labelled diagrams and compare classifications.

Before examination

Use the module questions, formula bank, official assessment pattern and final checklist. Treat the practice paper as preparation, not as an official question paper.



Use this cycle for every module: understand the official concept, represent it accurately, practise, and connect it to an application.

OUTCOMES AND ALIGNMENT

Objectives, course outcomes and CO-PO mapping

Official course objectives

1. Apply logical thinking and problem-solving techniques to real-world problems.
2. Write efficient and structured C programs using appropriate language constructs.
3. Build foundations for advanced programming languages.
4. Understand basic data structures and memory concepts.

Course outcomes

CO	Official course outcome	Bloom
CO1	Make use of sequential, conditional and looping structures in C.	Apply
CO2	Develop programs using arrays, matrices and strings.	Apply
CO3	Develop problem-solving skills using functions and structures.	Apply
CO4	Study pointers, files and dynamic memory allocation and develop programs using pointers and files.	Apply

CO-PO course articulation matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	2	3	3	3	-	-	2	-	-	-	2
CO2	3	3	3	3	-	-	2	-	-	-	2
CO3	3	3	3	3	-	-	2	-	-	-	2
CO4	3	3	3	3	-	-	2	-	-	-	2

Legend: 1 = Low; 2 = Medium; 3 = High; - = No Correlation.

NOTHING OMITTED

Complete syllabus coverage

Complete syllabus coverage map

Module	Official title	Instructional hours	Major syllabus coverage
I	Basics of C Programming & Control Statements	17 L	Character set, data types, operators, program structure, I/O, decisions, loops and transfer
II	Arrays and Strings	15 L	1D/2D arrays, searching, sorting, matrices and strings
III	Functions and Structures	15 L	Functions, recursion, structures and unions
IV	Pointers and File Handling	13 L	Pointers, pointer parameters, files, modes, opening, closing, read/write

Every official module and major topic is expanded below. The wording may be grouped for teaching clarity, but no supplied syllabus item is intentionally removed.

Official topic term: C fundamentals include *constants* as well as variables, identifiers, keywords, tokens, data types, declarations and initialization. Constants may be literal values or named values whose value is not changed during the intended computation.

LEARNING ROADMAP

How the modules connect

Module 1
Syntax & control

Module 2
Arrays & strings

MODULE 1

Basics of C Programming & Control Statements

C syntax turns a problem-solving plan into a structured executable program.

Official hours: 17 L

Mapped CO: CO1

Bloom level: Understand

Learning goals

- Identify tokens, data types, variables and operators.
- Write simple input/output programs.
- Select decision, iteration and transfer statements.

Key facts

- C is case-sensitive.
- A declaration introduces type and name.
- Choose loop type from the required repetition behavior.

Engineering relevance

Use the concepts to interpret technical information, communicate decisions accurately, and solve appropriately scoped problems.



A C program is written as source, translated by the toolchain, and executed with input and output.

Character set, tokens, identifiers, keywords, data types and variables–

The C character set supplies letters, digits and special symbols. A token is a meaningful lexical unit such as a keyword, identifier, constant, string literal or operator. Identifiers must follow naming rules and keywords cannot be used as user-defined names. Common data types include integer, floating-point, character and void forms.

Malayalam support: C case-sensitive: `Total` and `total` are different identifiers. Variable declaration requires a declaration.

```
int count = 0;
float voltage = 12.5f;
char grade = 'A';
```

Operators, precedence and expressions–

Arithmetic, relational, logical, conditional, unary, increment/decrement and assignment operators combine values into expressions. Precedence determines grouping, but parentheses should be used when they improve clarity. Distinguish assignment `=` from equality test `==`.

Given: int a=5, b=2.

Required: Evaluate a/b and a%b.

Calculation: Integer division gives 2; remainder gives 1.

Answer: a/b=2 and a%b=1.

Structure of C programs and printf/scanf–

A basic program commonly contains headers, a main() function, declarations, executable statements and a return statement. Use format specifiers that match the variable type. `printf()` displays formatted output; `scanf()` reads input and generally needs the address operator for ordinary variables.



A C program is written as source, translated by the toolchain, and executed with input and output.

```
#include <stdio.h>
int main(void){
    int n;
    scanf("%d", &n);
    printf("%d", n);
    return 0;
}
```

Decision, iterative and control-transfer statements–

Use if, if-else, nested if and switch for decisions. Use while when the condition is tested before each iteration, do-while when one execution is required before testing, and for for a compact count-controlled loop. Nested loops are useful for tables and matrices. `break` exits a loop/switch; `continue` skips to the next iteration.

Common mistake: Ensure the loop control variable changes and use braces when more than one statement belongs to a branch.

MODULE 2

Arrays and Strings

Arrays store related values under one name; strings are character arrays terminated by a null character.

Official hours: 15 L

Mapped CO: CO2

Bloom level: Understand

Learning goals

- Declare, initialize and access 1D and 2D arrays.
- Apply arrays to search, sort and matrix representation.
- Read, display and process strings with library functions.

Key facts

- C array indices start at 0.
- Do not access outside bounds.
- A C string ends with `.`.

Engineering relevance

Use the concepts to interpret technical information, communicate decisions accurately, and solve appropriately scoped problems.

Single-dimensional arrays–

A one-dimensional array has a fixed element type and contiguous indexed storage. Declare it with a size or initializer, access $A[i]$, and traverse it using a loop. Applications include sum, average, maximum, minimum, linear search and sorting.

```
int a[5] = {4, 1, 7, 2, 9};
int sum = 0;
for(int i=0;i<5;i++) sum += a[i];
```

Malayalam support: Array index 0 ന്റെ മൂല്യം 4 ആണ്. `a[5]` ന്റെ declaration-ന് valid indices 0 ന്റെ 4 മൂല്യം.

Multidimensional arrays and matrices–

A two-dimensional array represents rows and columns, such as a matrix. Use nested loops for initialization, display and matrix addition. Keep row and column limits explicit.

Given: $A = [[1,2],[3,4]]$, $B = [[5,6],[7,8]]$.

Required: $A+B$.

Calculation: Add corresponding entries.

Answer: $[[6,8],[10,12]]$.

Strings and library functions–

Declare a string as a character array with enough space for the terminating null character. The syllabus requires reading, displaying and string-related programs using library functions. Know the purpose and safe use of common functions such as length, copy, concatenate and compare; always consider buffer size and input method.

MODULE 3

Functions and Structures

Functions divide programs into reusable units, while structures and unions group related data.

Official hours: 15 L

Mapped CO: CO3

Bloom level: Apply

Learning goals

- Write prototypes, definitions and calls.
- Distinguish pass by value and passing addresses/reference-style behavior.
- Use recursion, structures and unions.

Key facts

- A prototype communicates a function signature.
- C passes arguments by value; pointers can allow a function to modify caller data.
- A union shares storage among members.

Engineering relevance

Use the concepts to interpret technical information, communicate decisions accurately, and solve appropriately scoped problems.

Functions and their components–

A function definition contains return type, name, parameters and body. A prototype declares the interface before use; a function call transfers control and supplies arguments. Built-in/library functions are provided by headers, while user-defined functions are written by the programmer.

```
int add(int a, int b);
int add(int a, int b){ return a+b; }
int main(void){ printf("%d", add(2,3)); }
```

Parameter passing and arrays–

Pass by value copies a value, so changing the parameter does not change the caller's ordinary variable. Passing an address through a pointer allows the called function to update caller storage. Array arguments provide access to the original array elements; document size assumptions.

Recursion–

Recursion is a function calling itself on a smaller subproblem. Every recursive solution needs a base case and progress toward that case.

```
int factorial(int n){
    if(n<=1) return 1;
    return n*factorial(n-1);
}
```

Malayalam support: Recursion- base case function calls . call- problem size .

Structures and unions–

A structure groups members of possibly different types and gives each member separate storage. Access members with the dot operator, or arrow through a pointer. A union declares different members sharing the same memory area; only the currently meaningful member should be read. The syllabus requires definition, declaration, member access and structure–union comparison.

Structure and union comparison

Basis	Structure	Union
Storage	Separate storage for each member	Members share storage
Use	Record with simultaneous fields	One of several representations at a time
Access	Dot/arrow operators	Dot/arrow operators

MODULE 4

Pointers and File Handling

Pointers expose addresses and enable indirect access, while files preserve data beyond one program execution.

Official hours: 13 L

Mapped CO: CO4

Bloom level: Apply

Learning goals

- Declare and safely dereference pointers.
- Pass pointers to functions.
- Open, read, write and close text files using modes.

Key facts

- Initialize pointers before dereferencing.
- Check file-open failure.
- Always close an opened file.

Engineering relevance

Use the concepts to interpret technical information, communicate decisions accurately, and solve appropriately scoped problems.

Pointers and pointer parameters–

A pointer stores the address of an object. `&x` obtains an address and `*p` dereferences it. Passing a pointer to a function permits indirect modification of caller data and is central to swap and array-processing programs.

```
void swap(int *a,int *b){
    int t=*a; *a=*b; *b=t;
}
```

Safety: Never dereference an uninitialized, null or invalid pointer; do not return the address of a local object that no longer exists.

Files, modes and operations–

A file is a named stream of stored data. Use a `FILE*` file pointer with `fopen()`, perform reading or writing, and call `fclose()`. The required modes are `r`, `w`, `a`, `r+`, `w+` and `a+`. Understand that write mode can truncate an existing file.

Basic C file modes

Mode	Purpose
<code>r</code>	Read existing file
<code>w</code>	Write; create or truncate
<code>a</code>	Append; create if needed
<code>r+</code>	Read and write existing file
<code>w+</code>	Read/write; create or truncate
<code>a+</code>	Read and append; create if needed

Opening, reading, writing, closing and troubleshooting–

Check the return value of `fopen()` before use. Use appropriate formatted or character/block operations for the file content. A complete solution handles errors, performs the requested operation and closes the file. Common errors include wrong mode, missing `fclose()`, using a file pointer before opening and confusing a file pointer with an ordinary data pointer.

```
FILE *fp = fopen("notes.txt","w");
if(fp == NULL) return 1;
fprintf(fp,"Hello\n");
fclose(fp);
```

RAPID CALCULATION REFERENCE

Formula bank

Array address idea

Address of $A[i] = \text{base} + i \times \text{element size}$

For a simple 1D array, indexing is zero-based in C; the actual byte calculation depends on type size.

Pointer dereference

$*p$ gives the object stored at the address held by p

p must point to a valid object before dereferencing.

File opening

`FILE *fp = fopen("name","mode");`

Check fp for NULL and close an opened file using `fclose(fp)`.

Average

$\text{average} = \text{sum} / \text{count}$

Use a suitable numeric type to avoid unintended integer truncation.

RAPID VISUAL REVISION

Diagram library

Use these labelled schematics to revise relationships and redraw them in examination answers where relevant.



A C program is written as source, translated by the toolchain, and executed with input and output.



Use this cycle for every module: understand the official concept, represent it accurately, practise, and connect it to an application.

OFFICIAL SELF-LEARNING

Activities and independent study

Suggested student activities for self-learning — wording retained in substance from the source PDF

Week / group	Official self-learning activity	Hours
1-3	Data-type table; execution steps/online compilers; basic-concepts quiz.	6
4-6	Algorithms and simple C programs with output screenshots; predict output/errors.	12
7	Array statistics programs and memory-organization drawing.	6
8-9	String programs, bubble-sort iterations, linear search, matrix addition and array quiz.	14
10-11	Function programs, output prediction, recursion, structures and storage classes.	18
12-14	Pointer arguments, file modes, incorrect-code correction, FILE pointer and pointer tracing.	30
15	File-operation program and Google Form quiz.	6

Submission tip: Keep a dated record of sketches, calculations, code/output screenshots, case-study evidence or field observations as appropriate. Use the exact activity title from the syllabus when submitting.

EXAMINATION READINESS

Assessment methodology

Theory assessment structure shown in the supplied Revision 2026 syllabus PDF.

Component	Official mode / duration	Marks / timing
Attendance	Attendance; end of semester	5 converted marks
CA1	Self-learning activities, Module 1; by week 4	15
CA2	Self-learning activities, Module 2; by week 7	15
CA3	Self-learning activities, Module 3; by week 11	15
CA4	Self-learning activities, Module 4; by week 14	15
CA5	Series examination, two modules; 2 hours; week 8	50 raw → 20 converted
CA6	Series examination, two modules; 2 hours; week 15	50 raw → 20 converted
ESE	Written theory; 2.5 hours; end of semester	60

Series examination pattern — theory

Part	Question pattern	Marks
Part A	4 questions total; 2 from each selected module; 1 mark each	4
Part B	6 questions total; 3 from each selected module; 3 marks each	18
Part C	4 questions total; 2 with choice from each selected module; 7 marks each	28
Total	Two selected modules, 2 hours	50

End Semester Examination pattern — theory

Module	Part A	Part B	Part C	Module total
1	$2 \times 1 = 2$	2 of $3 \times 3 = 6$	1 set with choice $\times 7$	15
2	2	6	7	15
3	2	6	7	15
4	2	6	7	15
Total	8	24	28	60

Important: The practice pattern below is generated for revision and must not be represented as an official examination paper.

MODULE-WISE QUESTION BANK

Questions and answers

Answers are deliberately matched to the likely mark value: definitions are concise, while longer answers use structure, comparison, formulae or diagrams.

Module I — C basics

What is a token?+

Differentiate = and ==.+

When is a do-while loop useful?+

Module II — Arrays and strings

What is an array?+

Why does a C string need a null character?+

Name two array applications in the syllabus.+

Module III — Functions and structures

What is a function prototype?+

What is recursion?+

Give one difference between structure and union.+

Module IV — pointers and files

What is a pointer?+

Why check fopen() result?+

Why is fclose() required?+

PRACTICE ONLY

Practice Model Paper — Not an Official Examination Paper

Series examination pattern — theory

Part	Question pattern	Marks
Part A	4 questions total; 2 from each selected module; 1 mark each	4
Part B	6 questions total; 3 from each selected module; 3 marks each	18
Part C	4 questions total; 2 with choice from each selected module; 7 marks each	28
Total	Two selected modules, 2 hours	50

End Semester Examination pattern — theory

Module	Part A	Part B	Part C	Module total
1	$2 \times 1 = 2$	2 of $3 \times 3 = 6$	1 set with choice $\times 7$	15
2	2	6	7	15
3	2	6	7	15
4	2	6	7	15
Total	8	24	28	60

LAST-MINUTE REVIEW

Quick revision and exam checklist

Module-wise key points

1. **Module 1:** Tokens, data types, operators, I/O, decisions, loops, break and continue.
2. **Module 2:** 1D/2D arrays, searching, sorting, matrices and null-terminated strings.
3. **Module 3:** Function prototype/definition/call, parameter passing, recursion, structures and unions.
4. **Module 4:** Pointers, safe dereference, file modes, fopen/fclose and read/write.

Exam checklist

- Write definitions before explanations.
- Use standard symbols, units, labels and terminology.
- For comparisons, use a table with a clear basis.
- For numerical work, show Given, Required, Calculation and Answer.
- For diagrams, include a title, labels and a one-line interpretation.
- Check that the answer addresses the command word: define, explain, compare, apply or discuss.

TECHNICAL VOCABULARY

Glossary

Token

Meaningful lexical unit in a C program.

Identifier

Programmer-defined name.

Array

Indexed collection of same-type elements.

String

Null-terminated character sequence.

Prototype

Function declaration describing its interface.

Recursion

Self-calling function technique with a base case.

Pointer

Object storing an address.

File pointer

A FILE* handle used for C stream operations.

Union

User-defined type whose members share storage.

SOURCE-SUPPORTED RESOURCES

Official learning resources

Only resources listed in the supplied syllabus PDF are included here.

- Textbook: *Programming in ANSI C*, E. Balagurusamy, McGraw-Hill.
- Textbook: *The C Programming Language*, Kernighan & Ritchie, Pearson.
- Reference: *C Programming Absolute Beginner's Guide*, Greg Perry, Pearson.

- Reference: *Let Us C*, Yashavant Kanetkar, BPB.
- Web: [GeeksforGeeks C Programming](#).
- Web: [TutorialsPoint C Programming](#).